



Lost & Found Final Draft Report Group:6

Team Member:

Student Name	ID	Section
Hajer Sayah	445013975	1
Zahraa Alsatif	445013956	1
Danah Alharbi	445000673	1
Lujain Zahid	445014050	2
Mayas AbuAlfarag	445006876	2

Task distribution:

Student Name	Task
Hajer Sayah	Proposed Methodology & Implementation - Evaluation Metercis
Zahraa Alsatif	Related Work
Danah Alharbi	Conclusion and Future Work
Lujain Zahid	Experimental Results and Discussion - Error Analysis
Mayas AbuAlfarag	Introduction, addressing problem gaps and the transfer learning methodology

Supervisor: Dr. Seereen Noorwali

Table of Contents

1. Introduction	3
1.1 Overview	3
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Research Gaps	3
1.5 Our Proposed Solution	3
2. Related Work	4
3. Proposed Methodology & Implementation	5
3.1 System Architecture	5
3.2 Dataset Description	6
3.3 Video Input Processing	6
3.4 Object Detection Stage	4
3.5 Object Tracking Stage	6
3.6 Data Structuring Stage	6
3.7 Spatial Analysis Stage	7
3.8 Behavioral Analysis Stage	7
3.8.1 Ownership Assignment	7
3.8.2 Forgotten Object Detection	7
3.8.3 Suspicious Behavior Detection	7
3.9 Evaluation Metrics	8
4. Experimental Results and Discussion	9
4.1 Quantitative Results	9
4.1.1 System Performance Metrics	9
4.1.2 Object State Classification Results	9
4.1.3 Standard Classification Metrics	10
4.1.4 Detected Events Table	10
4.2 Visualization Results	11-15
4.3 Discussion	16
4.4 Error Analysis	16
5. Conclusion and Future Work	17
5.1 Conclusion	17
5.2 Future Work	17
References	18-19

1. Introduction

1.1 Overview

SmartTrack is an intelligent Computer Vision system that enhances traditional surveillance by analyzing human-object interactions. It integrates YOLOv8 and ByteTrack to monitor personal belongings and identify security threats in real-time.

1.2 Problem Statement

Standard surveillance systems can detect presence but fail to interpret relationships. They cannot distinguish between a person intentionally leaving an item, accidentally forgetting it, or a potential theft, leading to security gaps and high false-alarm rates.

1.3 Objectives

- **Automate** detection and tracking of persons and personal items.
- **Determine** ownership through Euclidean distance-based proximity logic.
- **Classify** object states as Safe, Forgotten, or Suspected based on behavioral patterns.

1.4 Research Gaps

Existing solutions often require massive, environment-specific datasets and high computational power. Furthermore, many systems lose track of object ownership during crowded scenes or overlaps (occlusions), breaking the interaction chain.

1.5 Our Proposed Solution

Our strategy leverages Transfer Learning by utilizing YOLOv8 with pre-trained COCO weights, which allows for immediate, high-accuracy detection of essential classes like persons and various types of luggage. This approach is strategically chosen to bypass the need for large-scale fine-tuning on datasets such as ABODA and CAVIAR, which lack the extensive annotations required for deep learning training and often lead to overfitting in limited environments. By using these pre-trained weights, we maintain a resource-efficient pipeline that preserves generalized detection capabilities across diverse surveillance settings. Consequently, ABODA and CAVIAR are repurposed as strictly evaluative benchmarks to validate our behavioral logic and state classification, ensuring the system remains robust and scalable without the prohibitive computational costs of custom training.

2. Related Work

Recent advances in computer vision and intelligent surveillance have significantly improved abnormal event detection through the integration of deep learning-based object detection and multi-object tracking techniques. Many modern surveillance systems rely on real-time object detection models and behavioral analysis to monitor public environments and identify suspicious activities automatically [9], [13].

Several previous studies demonstrated the effectiveness of YOLO-based architectures for detecting objects in crowded and dynamic environments. YOLOv8, in particular, has shown strong performance in terms of detection accuracy and processing speed, making it suitable for real-time surveillance applications [1], [11]. These capabilities are important in surveillance scenarios where fast and continuous analysis of multiple moving objects is required.

In addition to object detection, multi-object tracking plays an essential role in maintaining object identities across consecutive video frames. Tracking approaches such as ByteTrack improve tracking robustness by associating detections over time while handling temporary occlusion and low-confidence detections [2], [4], [16]. This enables surveillance systems to follow the movement of persons and belongings consistently throughout a video sequence.

Research on abandoned object detection has also received considerable attention in recent years. Earlier approaches mainly relied on background subtraction, frame differencing, and motion analysis techniques to identify unattended objects in surveillance scenes [12], [14]. Other studies focused on extracting stable foreground regions and analyzing object persistence over time to improve unattended luggage detection accuracy [15].

Review studies further emphasized the importance of combining object detection, object tracking, and behavioral interpretation in intelligent surveillance systems [9], [13]. These studies highlighted that relying solely on object detection is insufficient for understanding complex human–object interactions or recognizing abnormal events accurately.

Inspired by these previous studies, the proposed system integrates object detection, multi-object tracking, spatial analysis, and rule-based behavioral logic into a unified framework. Unlike traditional approaches that focus only on detecting objects, the proposed method interprets interactions between people and belongings over time and classifies objects into meaningful behavioral states such as Safe, Forgotten, and Suspicious. This allows the system to move beyond simple visual detection toward intelligent event understanding in real-world surveillance environments.

3. Proposed Methodology & Implementation

3.1 System Architecture

The proposed system works as an end-to-end pipeline that starts with video input and ends with interpreting what happened to an object. The main idea is to move beyond simple object detection and toward understanding events in a scene. First, a video stream (live or recorded) is provided to the system. A deep learning model such as YOLOv8 is used to detect two main categories in each frame: people and personal belongings (such as backpacks or laptops).

After detection, the results are passed to a multi-object tracking module such as ByteTrack. In this stage, each detected person and object is assigned a unique ID so their movement can be followed across frames. This allows the system to keep track of who interacted with which object over time, even when temporary occlusion or movement occurs. Once tracking information is available, the system analyzes human–object interactions using simple rule-based logic.

For example, when a person places an object and walks away, the system marks that person as the potential owner. If the owner stays beyond a set distance for a specific time, the object becomes "unattended." If another person then approaches, picks it up, and leaves, the event is labeled as "Suspicious Pickup." Based on this, each object is classified as Safe, Forgotten, or Suspicious Pickup. Finally, the system displays bounding boxes, status labels, and a short event summary on a dashboard.

The overall architecture of the proposed system is illustrated in Figure 1. The system follows a sequential pipeline starting from video input, followed by object detection, multi-object tracking, human–object interaction analysis, state classification, and finally alert generation and event reporting through a dashboard.

When the system classifies an object as "Forgotten" or "Suspicious Pickup", it automatically generates an alert. The alert is displayed on the dashboard along with the bounding boxes, status labels, and event summary, ensuring that security personnel are notified immediately of potential incidents.

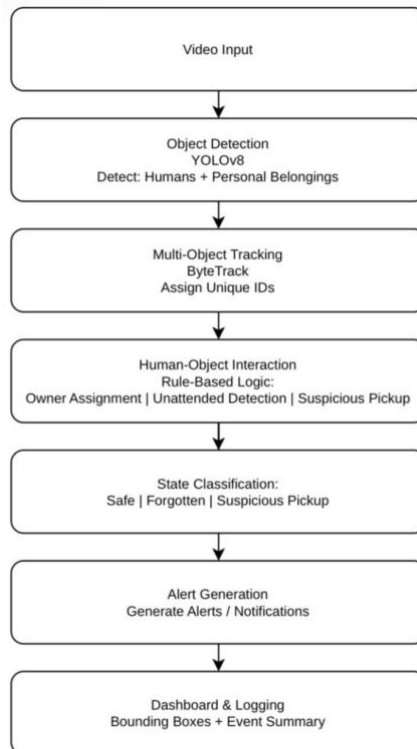


Figure 1: System Architecture of the Proposed Lost & Found Vision System

3.2 Dataset Description

We used two publicly available surveillance datasets for testing. The first is the CAVIAR dataset from the University of Edinburgh [7]. It contains videos of abandoned luggage scenarios, including LeftBag.mpg, LeftBox.mpg, and LeftBag_PickedUp.mpg. We used 5 videos from this dataset, each with a resolution of 384×288 pixels (half-resolution PAL), in color (RGB) format at 25 frames per second. These videos were recorded in an indoor laboratory environment simulating a public lobby.

The second is the ABODA (Abandoned Objects Dataset) [6]. It contains abandoned object scenarios in public settings such as train stations and sidewalks. We used 11 videos from this dataset, each with a resolution of 640×480 pixels, in color (RGB) format, recorded in outdoor public spaces.

Since we used a pre-trained YOLOv8 model (trained on the COCO dataset), no additional training was performed on these datasets. They were used exclusively for testing the detection, tracking, and interaction logic. A total of 2,189 frames from video1.avi were processed for quantitative evaluation.

3.3 Video Input Processing

These videos are read using OpenCV [5]. Then, the videos are segmented into individual frames and processed independently. This process is required for the detection of any variations in the scene at different moments, such as the movement of individuals and other items .

3.4 Object Detection Stage

In this stage the YOLOv8[1] model is applied to detect objects of interest. The model identifies persons and carries objects and returns bounding boxes for each detection. A confidence threshold of 0.15 is applied to filter out low-confidence detections, which improves detection on older or lower-quality videos. These detections are the basis of all subsequent analysis. Output of this stage is a list of detected entities per frame with spatial coordinates and class labels.

3.5 Object Tracking Stage

The detected objects are then passed to a tracker module based on the ByteTrack algorithm [2][4]. ByteTrack is configured using a custom 'bytetrack.yaml' file [5] with modified thresholds ('track_high_thresh=0.25', 'track_low_thresh=0.05', 'new_track_thresh=0.25'). The 'persist=True' parameter ensures that each object retains the same ID across consecutive frames. This enables the system to track objects when they are temporarily occluded from view.

3.6 Data Structuring Stage

After tracking, the system organizes all detected entities into a structured format. Each object is represented using:

- Unique ID
- Object type (person/object)
- Bounding box coordinates

This structured representation allows efficient processing in the behavioral analysis stage and ensures that all temporal data is stored in an accessible format.

3.7 Spatial Analysis Stage

The system calculates spatial relationships between persons and objects using Euclidean distance between bounding box centers. This distance measurement is used to determine proximity and interaction levels between entities. It is a key factor in identifying ownership and detecting abnormal separation between a person and their associated object .

Thresholds Used in Implementation

The following numerical thresholds were used in the behavioral logic:

Parameter	Value
Owner distance threshold	120 pixels
Unattended distance threshold	180 pixels
Unattended frames threshold	30 frames
Suspicious pickup distance threshold	100 pixels

3.8 Behavioral Analysis Stage

The final step is to interpret interactions over time using rule-based logic. Each object is assigned a state that changes according to its relationship to nearby persons. The system identifies three majors behavioral states:

- Safe: object is near to its owner
- Forgotten: An unattended object for some time
- Suspected: Unauthorized interaction detected

This stage transforms low-level tracking data into semantically meaningful events.

3.8.1 Ownership Assignment

At the beginning of tracking, each object is assigned to the closest detected person. This person is considered as the object's owner. The system continuously monitors the distance between the object and the owner to determine whether the object remains attended or becomes separated over time.

3.8.2 Forgotten Object Detection

An object is classified as forgotten once their owner moves beyond a defined spatial threshold for a continuous number of frames. This makes sure that that temporary movement does not trigger false alarms. Only sustained separation leads to a change in object state.

3.8.3 Suspicious Behavior Detection

If a different person approaches a forgotten object, the system identifies their interaction as suspicious. This logic helps detect potential theft or unauthorized handling of unattended objects by analyzing both spatial proximity and object state history.

3.9 Evaluation Metrics

To evaluate the system, we rely on metrics that reflect our actual implementation.

Detection and Tracking Performance

Although standard metrics such as mAP@0.5 and MOTA are commonly used in object detection and tracking, they were not computed due to the lack of ground truth annotations in the selected datasets (CAVIAR and ABODA).

Behavioral Analysis Metrics

The most important metrics for our project are:

- State Classification Accuracy: Measures how correctly the system classifies objects as Safe, Forgotten, or Suspicious.
- Precision, Recall, F1-Score: Used to evaluate the detection of positive events such as Forgotten Object and Suspicious Pickup, where True Positive indicates correct identification of a dangerous or unattended situation.

System Performance Metrics

We use the following metrics to evaluate system efficiency:

- FPS (Frames Per Second): Measures processing speed.
- Processing Time: Total time to process a video.
- Unique IDs: Number of unique persons and objects detected (validates tracking consistency) .

4. Experimental Results and Discussion

4.1 Quantitative Results

4.1.1 System Performance Metrics

The quantitative results presented in this section were obtained from a representative test video (**video1.avi**) containing 2,189 frames. The proposed system processed the entire video in 21.95 seconds and achieved an average speed of approximately 99.73 frames per second. This high processing speed demonstrates that the system is capable of near real-time operation. In addition, the system successfully tracked 17 unique persons and 26 unique objects, confirming its ability to maintain object identities and analyze human–object interactions throughout the video sequence.

Metric	Result
FPS (Frames Per Second)	99.73
Total Frames Processed	2189
Total Processing Time	21.95 seconds
Unique Persons Detected	17
Unique Objects Detected	26

4.1.2 Object State Classification Results

The final object state classification results show that most detected objects were labeled as Safe, indicating that they remained associated with their owners during the video. One object was classified as Forgotten after being left unattended for a sufficient number of frames, while five objects remained in the Unknown state because no reliable owner could be assigned.

State	Count
Safe Objects	20
Forgotten Objects	1
Unknown Objects	5

4.1.3 Standard Classification Metrics (Precision, Recall, F1-Score)

To further evaluate the event detection performance, standard classification metrics were calculated using the detected events and the expected ground-truth events. The system measured Precision, Recall, and F1-Score to evaluate how accurately the system detected abnormal events such as Forgotten Object and Suspicious Pickup.

(These metrics were calculated using **video1.avi** because it contains clear ground-truth events for comparison.)

Metric	Value
Precision	1.0
Recall	0.5
F1-Score	0.67

4.1.4 Detected Events Table

The event detection results indicate that the system successfully assigned ownership to 21 objects and detected one Forgotten Object event. These results confirm that the rule-based interaction analysis module was able to associate objects with nearby persons and identify unattended belongings when the owner moved away.

Event	Count
Owner Assigned	21
Forgotten Object	1

4.2 Visualization Results

Figure 2: Object Detection Output

Figure 2 shows the output of the YOLOv8 object detection model. The system successfully detected persons and personal belongings and displayed bounding boxes with confidence scores.



Figure 3: Multi-Object Tracking Output

Figure 3 illustrates the ByteTrack tracking results, where each detected object is assigned a unique ID.



Figure 4: Object State Distribution

Figure 4 summarizes the final classification of detected objects into (20 Safe, 1 Forgotten, and 5 Unknown states).

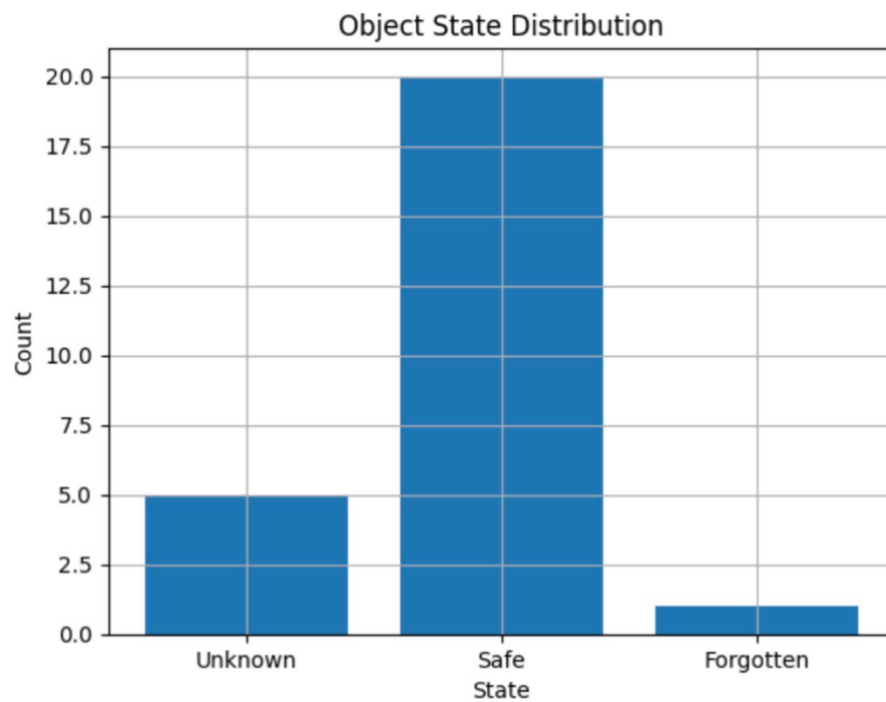


Figure 5: Detected Events Distribution

Figure 5 presents the number of detected events, including (21 ownership assignment and 1 forgotten object detection).



Figure 6: Interaction Logic Output

Figure 6 shows the console output generated by the rule-based interaction analysis module, including final object states and detailed event records.

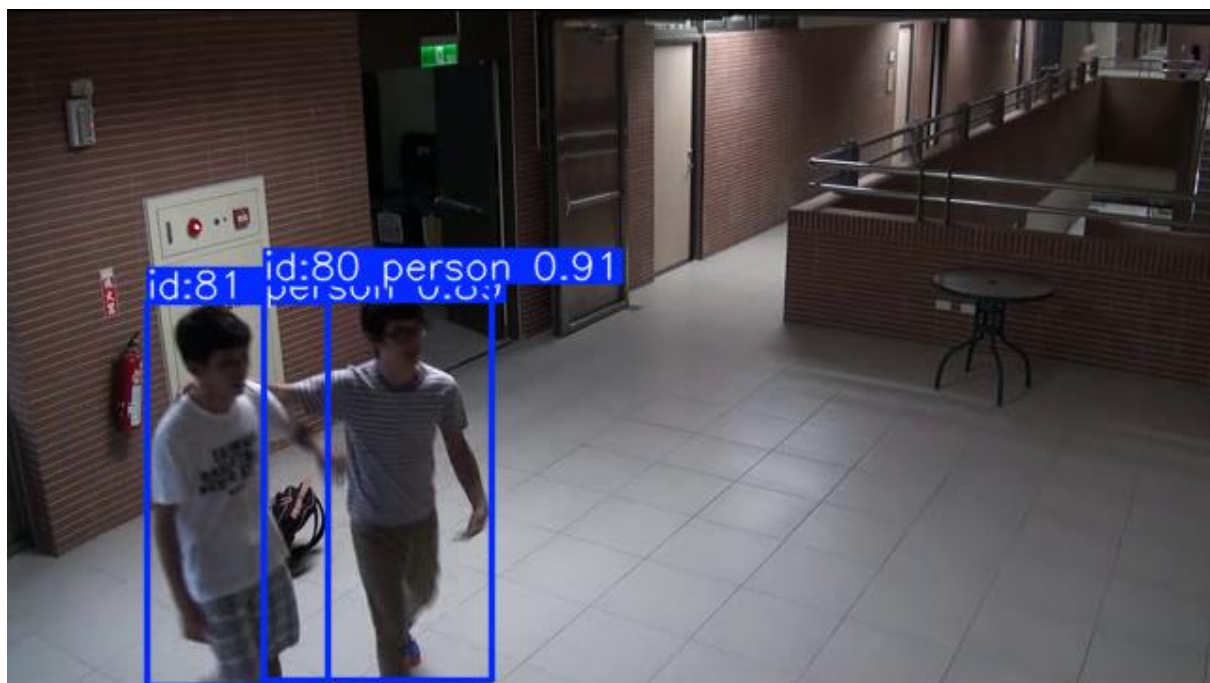
```
Tracking complete for 2189 frames.
Final Object States:
5 {'owner_id': None, 'state': 'Unknown', 'unattended_counter': 0, 'type': 'object'}
10 {'owner_id': None, 'state': 'Unknown', 'unattended_counter': 0, 'type': 'object'}
17 {'owner_id': 8, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
33 {'owner_id': 19, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
39 {'owner_id': 36, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
40 {'owner_id': 36, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
77 {'owner_id': 60, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
84 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
88 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
90 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
94 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
100 {'owner_id': 80, 'state': 'Forgotten', 'unattended_counter': 118, 'type': 'object'}
104 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
110 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
122 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
123 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
133 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
136 {'owner_id': 80, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
162 {'owner_id': 142, 'state': 'Safe', 'unattended_counter': 0, 'type': 'object'}
```

```
Detected Events:
{'frame': 260, 'object_id': 17, 'event': 'Owner Assigned', 'details': 'Person 8 assigned as owner of Object 17'}
{'frame': 330, 'object_id': 33, 'event': 'Owner Assigned', 'details': 'Person 19 assigned as owner of Object 33'}
{'frame': 594, 'object_id': 39, 'event': 'Owner Assigned', 'details': 'Person 36 assigned as owner of Object 39'}
{'frame': 594, 'object_id': 40, 'event': 'Owner Assigned', 'details': 'Person 36 assigned as owner of Object 40'}
{'frame': 861, 'object_id': 77, 'event': 'Owner Assigned', 'details': 'Person 60 assigned as owner of Object 77'}
{'frame': 1203, 'object_id': 84, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 84'}
{'frame': 1215, 'object_id': 88, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 88'}
{'frame': 1221, 'object_id': 90, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 90'}
{'frame': 1250, 'object_id': 94, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 94'}
{'frame': 1264, 'object_id': 100, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 100'}
{'frame': 1276, 'object_id': 104, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 104'}
{'frame': 1300, 'object_id': 110, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 110'}
{'frame': 1319, 'object_id': 122, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 122'}
{'frame': 1331, 'object_id': 123, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 123'}
{'frame': 1384, 'object_id': 133, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 133'}
{'frame': 1389, 'object_id': 136, 'event': 'Owner Assigned', 'details': 'Person 80 assigned as owner of Object 136'}
{'frame': 1458, 'object_id': 100, 'event': 'Forgotten Object', 'details': 'Object 100 became unattended'}
{'frame': 1482, 'object_id': 162, 'event': 'Owner Assigned', 'details': 'Person 142 assigned as owner of Object 162'}
{'frame': 2014, 'object_id': 200, 'event': 'Owner Assigned', 'details': 'Person 194 assigned as owner of Object 200'}
{'frame': 2021, 'object_id': 206, 'event': 'Owner Assigned', 'details': 'Person 194 assigned as owner of Object 206'}
```

Results on(video1.avi) The system processed a video of 2,189 frames and tracked multiple objects throughout the sequence, assigning some objects to specific owners while others remained unassigned. Each object was classified into states such as Safe (continuously with its owner), Unknown (no reliable owner identified), or Forgotten (left unattended for a period). Most objects remained in a Safe state, meaning they stayed with their assigned person without issues. A few objects were labeled Unknown due to missing or uncertain ownership, and at least one object was marked as Forgotten after being left unattended for a significant number of frames. The system also recorded key events showing when ownership was assigned and when an object became unattended during the video.

Figure 7: Forgotten Object Detection Example

Figure 7: Example of a forgotten object scenario. The sequence illustrates a person leaving a personal belonging unattended, after which the system classifies the object as Forgotten and generates an alert.





This example demonstrates how the proposed system analyzes object ownership over time and detects unattended belongings when the assigned owner moves beyond the defined spatial and temporal thresholds.

4.3 Discussion

The experimental results demonstrate that the proposed system was effective in detecting people and personal belongings and in analyzing human-object interactions. The Precision, Recall, and F1-Score values indicate that the system was able to detect abnormal events with high precision and moderate recall. The system correctly assigned object ownership and detected unattended objects when owners moved away for a sufficient number of frames. Furthermore, the high FPS value confirms the suitability of the proposed approach for real-time surveillance applications. Overall, the results show that the integration of YOLOv8, ByteTrack, and rule-based interaction logic provides an effective solution for automated lost-and-found monitoring. The visual outputs and statistical charts further support the quantitative results by illustrating the distribution of object states and detected events throughout the analyzed video.

4.4 Error Analysis

Several limitations were observed during the experiments. In some cases, small or partially occluded objects were not detected correctly. Tracking performance was affected when people temporarily disappeared or overlapped with other objects. Lighting variations and low-resolution video frames also reduced detection confidence in certain scenes. The YOLOv8 model was used in its pre-trained form without additional fine-tuning on the selected datasets, which may have limited detection performance. Standard metrics such as mAP@0.5 and MOTA were not computed because the ABODA and CAVIAR datasets required substantial preprocessing and conversion of ground-truth annotations before they could be integrated with the YOLOv8 and ByteTrack outputs. In addition, PETS2007 dataset was initially considered because it provides richer ground-truth annotations suitable for formal evaluation. However, at the time of this project, the public download link was no longer accessible, so the ABODA and CAVIAR datasets were used instead. Despite these limitations, the system was able to demonstrate stable performance and successfully detect the main lost-and-found scenarios considered in this project.

5. Conclusion and Future Work

5.1 Conclusion

This project presented SmartTrack, an intelligent surveillance system designed to analyze human–object interactions using computer vision techniques. The proposed system combined YOLOv8 object detection, ByteTrack multi-object tracking, and rule-based behavioral analysis to monitor personal belongings and identify abnormal situations such as forgotten objects and suspicious pickups.

The experimental results demonstrated that the system was capable of detecting and tracking multiple persons and objects in surveillance videos with high processing speed. The system achieved approximately 99.73 FPS while processing 2,189 frames, confirming its suitability for near real-time surveillance applications. In addition, the behavioral analysis module successfully classified objects into Safe, Forgotten, and Unknown states and generated alerts when unattended objects were detected.

The integration of spatial analysis and interaction logic allowed the system to move beyond traditional object detection toward intelligent scene understanding. Unlike standard surveillance systems that only identify object presence, SmartTrack interpreted relationships between people and belongings over time, enabling more meaningful event detection.

Although some limitations were observed in crowded scenes, occlusions, and low-resolution videos, the overall results confirmed the effectiveness of the proposed framework for automated lost-and-found monitoring in public environments.

5.2 Future Work

Several improvements can be explored in future work to enhance the proposed system further.

- Fine-tuning the YOLOv8 model on surveillance-specific datasets could improve detection accuracy for small or partially occluded objects.
- Additional behavioral states such as object stealing, object exchange, or suspicious loitering could be incorporated to support more advanced surveillance analysis.
- More robust ownership assignment methods based on trajectory analysis and temporal interaction history could reduce incorrect ownership detection in crowded scenes.
- The system could be integrated with real-time CCTV cameras and cloud-based monitoring platforms to support large-scale deployment.
- Future work may also include the use of deep learning-based action recognition models instead of rule-based logic to improve event understanding and reduce dependence on manually selected thresholds.
- Formal evaluation using datasets with fully annotated ground-truth labels would allow the calculation of advanced metrics such as mAP, MOTA, and IDF1 for more comprehensive performance analysis.

Overall, the proposed SmartTrack system provides a strong foundation for intelligent surveillance applications and demonstrates the potential of combining object detection, tracking, and behavioral reasoning for automated security monitoring.

References

- [1] M. Yaseen, “What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector,” arXiv preprint arXiv:2408.15857, 2024. [Online]. Available: <https://arxiv.org/abs/2408.15857>. Accessed: May 2026.
- [2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, vol. 13682. Cham, Switzerland: Springer, 2022, pp. 1–21, doi: 10.1007/978-3-031-20047-2_1.
- [3] Ultralytics, “Ultralytics YOLOv8 Documentation,” 2026. [Online]. Available: <https://docs.ultralytics.com>. Accessed: May 2026.
- [4] Ultralytics, “Multi-Object Tracking – Ultralytics YOLOv8 Docs,” 2026. [Online]. Available: <https://docs.ultralytics.com/modes/track/>. Accessed: May 2026.
- [5] OpenCV Team, “Reading and Writing Videos using OpenCV,” 2026. [Online]. Available: <https://opencv.org/>. Accessed: May 2026.
- [6] K. Lin, “ABODA: Abandoned Object Dataset,” GitHub repository, 2015. [Online]. Available: <https://github.com/kevinlin311tw/ABODA>. Accessed: May 2026.
- [7] R. B. Fisher, “CAVIAR: Context Aware Vision using Image-based Active Recognition Dataset,” University of Edinburgh, 2004. [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/CAVIARDATA1/>. Accessed: May 2026.
- [8] D. P. Kumuda, B. V. Harshitha, K. Manikya, H. S. Dhavan, A. Ankita, and H. Y. Swathi, “A Comprehensive Review of AI-Powered Campus Surveillance: Trends, Challenges, and Future Directions from 2020 to 2025,” ITM Web Conf., vol. 81, p. 01017, 2026, doi: 10.1051/itmconf/20268101017.
- [9] W. Zhao, M. Mei, X. Wang, and H. Luo, “PKHOI: Enhancing Human-Object Interaction Detection Algorithms with Prior Knowledge,” Comput. Sci., vol. 53, no. 1, pp. 141–152, 2026, doi: 10.11896/jsjx.250100086.
- [10] D. Stiller, M. Wurm, J. Staab, T. Stark, G. Starz, J. Rauh, S. Dech, and H. Taubenböck, “Open Webcam Data for Traffic Monitoring: YOLOv8 Detection of Road Users Before and During COVID-19,” Transp. Res. Interdiscip. Perspect., vol. 36, p. 101774, 2026, doi: 10.1016/j.trip.2025.101774.
- [11] P. Yadav and A. Jahagirdar, “Performance Analysis of Machine Learning Classification Algorithms in Static Object Detection for Video Surveillance Applications,” Wireless Pers. Commun., vol. 115, pp. 2885–2904, 2020, doi: 10.1007/s11277-020-07627-1.
- [12] R. K. Tripathi, A. S. Jalal, and S. C. Agrawal, “Abandoned or Removed Object Detection from Visual Surveillance: A Review,” Multimedia Tools Appl., vol. 78, no. 6, pp. 7585–7620, 2019, doi: 10.1007/s11042-018-6472-9.
- [13] M. Din, A. Bashir, A. Basit, and S. Lakho, “Abandoned Object Detection Using Frame Differencing and Background Subtraction,” Int. J. Adv. Comput. Sci. Appl., vol. 11, no. 7, pp. 595–601, 2020, doi: 10.14569/IJACSA.2020.0110781.

[14] G. Szwoch, "Extraction of Stable Foreground Image Regions for Unattended Luggage Detection," *Multimedia Tools Appl.*, vol. 75, no. 2, pp. 761–786, 2016, doi: 10.1007/s11042-014-2324-4.